

Task 00 — Maya and Alaa — Define the stories and data structure

Members: Maya and Alaa

Deliverable: docs/data.js

Deadline: 2 October 2026, 23:59 — Damascus time

Context

Prepare **three mystery proposals, each with three ordered stages**, so the team can choose what to implement. This is content preparation, not building three games.

You may invent ideas, search online, use the instructor's examples, and ask others for feedback. The final stories and data structure are your responsibility.

Use docs/data.js for this phase; its temporary location is intentional. Keep the object name `mysteryData`, but define the final fields together. You may rename, add, remove, or rearrange the example fields:

```
// Example only. Your mission is to define the final structure.
// Fill it with three mysteries, each with three ordered stages.
const mysteryData = {
  mysteries: [
    {
      id: "",
      title: "",
      summary: "",
      story: "",
      stages: [
        {
          id: "",
          title: "",
          question: "",
          clues: [],
          answers: [],
          hints: [],
        },
      ],
      finalReveal: "",
    },
  ],
};
```

Goal

Create three complete mystery proposals and one agreed data structure that both members understand.

What to build

1. Give each mystery a unique, stable id, title, short summary, opening story, and player goal.
2. Write three stages per mystery: **nine stages total**. Each needs a question/challenge, enough clues, correct answer or accepted variants, and helpful hints.
3. Agree on the hint count. Two per stage is a suggestion. Hints should give more help step by step and match the clues.
4. Write a final reveal that resolves each story and connects its stages.
5. Use consistent fields and value types across mysteries, arrays for groups of items, and unique stage ids within each mystery. Explain stage order and any unclear fields in short comments.
6. Comment on answer rules: capitalization, surrounding spaces, and accepted spellings. Do not write validation code.
7. Fill all content. Add online source links where used, both names, contribution notes, and confirmation of your agreement in comments.

Rules

- Submit only `docs/data.js`. No API, frontend, database, or game logic.
- Use original story/clue wording as the specification requires. Adapt researched ideas instead of copying a complete game.
- Questions must be solvable from the supplied story and clues, without online searches.
- Keep stories short and answers clear enough to check. Avoid answers that need a personal judgment or long explanation.
- Keep data JSON-compatible: objects, arrays, strings, numbers, booleans, and `null` where needed; no functions.
- Both members use the same ids and structure, contribute, and understand all three stories.
- The file includes answers and endings. A future API must return only what the player should see at the current stage.

Test it

- Read each opening, its three stages, and its ending. Does the ending resolve the story?
- Try each other's questions using only the clues. Fix unclear questions and answers.
- Check every stage has questions, clues, answers, and useful hints.
- Check fields, types, ids, and stage order. Leave no missing content or example placeholders.
- From the repository root, run:

```
node --check docs/data.js
```

This checks JavaScript syntax only, not story logic or completeness.

- Compare both branch copies, including final corrections: they must match exactly.

Done when

`mysteryData` contains three complete proposals with three stages each and all required content. The structure and comments are clear, the syntax check passes, and both members have reviewed, agreed,

and independently pushed identical files for Majed's review.

Working time

Aim for **2–3 hours together**, including discussion and trying the puzzles. Keep the scope small and raise blockers early.

Submit before **2 October 2026, 23:59 — Damascus time**.

Submitting

Agree on the final file. Each member commits and pushes using their own Git identity. One push does not count for both. Do not push to **main**.

Maya, from the repository root on **member-maya**:

```
git add docs/data.js
git commit -m "Define agreed mystery stories and data structure"
git push origin member-maya
```

Alaa, from the repository root on **member-alaa**:

```
git add docs/data.js
git commit -m "Define agreed mystery stories and data structure"
git push origin member-alaa
```

Send Majed your branch's file link. Commit IDs may differ; file contents must match exactly. If you revise the agreement, both members update and push their copies.